

C++11新特性自学手册

目录

1. 基础语法增强
 2. 自动类型推导
 3. 智能指针
 4. 移动语义和右值引用
 5. Lambda表达式
 6. 类的新功能
 7. 模板改进
 8. STL容器和算法增强
 9. 并发编程基础
 10. 实战练习
-

1. 基础语法增强

1.1 统一初始化（列表初始化）

传统C++初始化的问题：

```
cpp

int x = 5;      // 基本类型
int arr[3] = {1,2,3}; // 数组
vector<int> v(5); // 容器（不直观）
```

C++11的解决方案：

```
cpp
```

```
// 使用大括号{}统一初始化
int x{5};
int y = {10};
double z{3.14};

// 数组
int arr[]{1, 2, 3, 4, 5};

// 容器
vector<int> v{1, 2, 3, 4, 5};
string s{"Hello World"};

// 对象
class Point {
public:
    Point(int x, int y) : x_(x), y_(y) {}
private:
    int x_, y_;
};

Point p{10, 20}; // 调用构造函数
Point p2 = {30, 40}; // 同样有效
```

防止类型收窄：

```
cpp

// 这些在C++11中会编译错误
char c1{1.5}; // double到char，编译错误
int i{3.14}; // double到int，编译错误

// 但这些是允许的
char c2{65}; // int到char，在范围内
double d{42}; // int到double，扩展转换
```

1.2 nullptr关键字

传统问题：

```
cpp

void func(int x) { cout << "int version" << endl; }
void func(char* p) { cout << "pointer version" << endl; }

func(0); // 调用int版本，可能不是预期的
func(NULL); // 可能有歧义
```

C++11解决方案：

```
cpp

func(nullptr); // 明确调用指针版本

int* p = nullptr; // 清晰的空指针
if (p == nullptr) {
    cout << "p is null" << endl;
}
```

1.3 强类型枚举

传统枚举的问题：

```
cpp

enum Color { RED, GREEN, BLUE };
enum Size { SMALL, MEDIUM, LARGE };

// 问题：名称冲突和隐式转换
// int x = RED; // 可以隐式转换为int
```

C++11强类型枚举：

```
cpp

enum class Color { RED, GREEN, BLUE };
enum class Size { SMALL, MEDIUM, LARGE };

Color c = Color::RED;    // 必须使用作用域
Size s = Size::LARGE;

// int x = Color::RED;    // 编译错误，不能隐式转换
int x = static_cast<int>(Color::RED); // 必须显式转换

// 指定底层类型
enum class Status : char { OK = 1, ERROR = 2 };
```

2. 自动类型推导

2.1 auto关键字

基本用法：

```
cpp
```

```

auto x = 42;    // int
auto y = 3.14;  // double
auto s = "hello"; // const char*
auto v = string("world"); // string

// 复杂类型简化
map<string, vector<int>> data;
// 传统写法
map<string, vector<int>>::iterator it = data.begin();
// C++11 写法
auto it = data.begin();

```

注意事项：

```

cpp

const int x = 10;
auto y = x;    // y是int，不是const int
auto& z = x;    // z是const int&

int arr[5];
auto a = arr;   // a是int*，不是int[5]
auto& b = arr;   // b是int(&)[5]

```

2.2 decltype关键字

获取表达式的类型：

```

cpp

int x = 10;
decltype(x) y = 20;    // y的类型是int

const int& func();
decltype(func()) z = x; // z的类型是const int&

// 与auto结合使用
template<typename T, typename U>
auto add(T t, U u) -> decltype(t + u) {
    return t + u;
}

```

decltype的规则：

```

cpp

```

```
int x = 10;
int& r = x;

decltype(x) // int
decltype(r) // int&
decltype((x)) // int& (加了括号变成引用)
```

3. 智能指针

3.1 unique_ptr - 独占所有权

```
cpp

#include <memory>

// 创建unique_ptr
unique_ptr<int> p1(new int(42));
auto p2 = make_unique<int>(42); // C++14推荐方式

// 访问
cout << *p1 << endl;
cout << *p2 << endl;

// 转移所有权
unique_ptr<int> p3 = move(p1); // p1变为nullptr
// unique_ptr<int> p4 = p2;    // 编译错误, 不能复制

// 自定义删除器
auto deleter = [](int* p) {
    cout << "Deleting " << *p << endl;
    delete p;
};
unique_ptr<int, decltype(deleter)> p4(new int(100), deleter);
```

使用示例：

```
cpp
```

```

class Resource {
public:
    Resource(int id) : id_(id) {
        cout << "Resource " << id_ << " created" << endl;
    }
    ~Resource() {
        cout << "Resource " << id_ << " destroyed" << endl;
    }
    void use() { cout << "Using resource " << id_ << endl; }
private:
    int id_;
};

void useResource() {
    auto resource = make_unique<Resource>(1);
    resource->use();
    // 函数结束时自动释放资源
}

```

3.2 shared_ptr - 共享所有权

```

cpp
// 创建shared_ptr
shared_ptr<int> p1(new int(42));
auto p2 = make_shared<int>(42); // 推荐方式，性能更好

// 共享所有权
shared_ptr<int> p3 = p2; // 引用计数变为2
cout << "引用计数: " << p2.use_count() << endl;

{
    shared_ptr<int> p4 = p2; // 引用计数变为3
    cout << "引用计数: " << p2.use_count() << endl;
} // p4离开作用域，引用计数变为2

// 当引用计数变为0时，自动删除对象

```

注意循环引用问题：

```

cpp

```

```

struct Node {
    shared_ptr<Node> next;
    weak_ptr<Node> parent; // 使用weak_ptr打破循环
    ~Node() { cout << "Node destroyed" << endl; }
};

auto node1 = make_shared<Node>();
auto node2 = make_shared<Node>();
node1->next = node2;
node2->parent = node1; // 使用weak_ptr，不会增加引用计数

```

4. 移动语义和右值引用

4.1 理解左值和右值

```

cpp

int x = 10;    // x是左值
int y = x + 5; // x是左值，x+5是右值
int z = 20;    // z是左值，20是右值

// 左值：有名字，可以取地址
int* px = &x; // OK

// 右值：临时对象，不能取地址
// int* py = &(x + 5); // 编译错误

```

4.2 右值引用

```

cpp

// 左值引用
int x = 10;
int& lr = x;    // 绑定到左值
// int& lr2 = 20; // 编译错误，不能绑定到右值

// 右值引用
int&& rr = 20;    // 绑定到右值
int&& rr2 = x + 5; // 绑定到右值
// int&& rr3 = x; // 编译错误，不能绑定到左值

// const左值引用可以绑定到右值
const int& clr = 30; // OK

```

4.3 移动构造函数和移动赋值


```
class MyString {
private:
    char* data_;
    size_t size_;

public:
    // 构造函数
    MyString(const char* str = "") {
        size_ = strlen(str);
        data_ = new char[size_ + 1];
        strcpy(data_, str);
        cout << "Constructor called" << endl;
    }

    // 拷贝构造函数
    MyString(const MyString& other) {
        size_ = other.size_;
        data_ = new char[size_ + 1];
        strcpy(data_, other.data_);
        cout << "Copy constructor called" << endl;
    }

    // 移动构造函数
    MyString(MyString&& other) noexcept {
        data_ = other.data_;
        size_ = other.size_;
        other.data_ = nullptr; // 重要：置空源对象
        other.size_ = 0;
        cout << "Move constructor called" << endl;
    }

    // 拷贝赋值运算符
    MyString& operator=(const MyString& other) {
        if (this != &other) {
            delete[] data_;
            size_ = other.size_;
            data_ = new char[size_ + 1];
            strcpy(data_, other.data_);
        }
        cout << "Copy assignment called" << endl;
        return *this;
    }

    // 移动赋值运算符
    MyString& operator=(MyString&& other) noexcept {
        if (this != &other) {
```

```

        delete[] data_;
        data_ = other.data_;
        size_ = other.size_;
        other.data_ = nullptr;
        other.size_ = 0;
    }
    cout << "Move assignment called" << endl;
    return *this;
}

~MyString() {
    delete[] data_;
    cout << "Destructor called" << endl;
}

const char* c_str() const { return data_ ? data_ : ""; }
};

// 使用示例
MyString createString() {
    return MyString("Hello"); // 返回临时对象
}

int main() {
    MyString s1("World");           // 构造函数
    MyString s2 = s1;               // 拷贝构造函数
    MyString s3 = createString();   // 移动构造函数

    s1 = MyString("New");           // 移动赋值
    s2 = s1;                       // 拷贝赋值

    // 强制移动
    MyString s4 = move(s2);         // 移动构造函数
    s1 = move(s3);                 // 移动赋值

    return 0;
}

```

4.4 std::move和完美转发

cpp

```
#include <utility>

// std::move: 强制转换为右值引用
vector<MyString> vec;
MyString s("Hello");
vec.push_back(move(s)); // s会被移动, 之后不应再使用

// 完美转发示例
template<typename T>
void wrapper(T&& arg) {
    func(forward<T>(arg)); // 保持参数的值类别
}
```

5. Lambda表达式

5.1 基本语法

```
cpp

// 基本形式: [捕获列表](参数列表) -> 返回类型 { 函数体 }

// 最简单的lambda
auto lambda1 = []() { cout << "Hello Lambda!" << endl; };
lambda1();

// 带参数
auto lambda2 = [](int x, int y) { return x + y; };
int result = lambda2(3, 4);

// 自动推导返回类型
auto lambda3 = [](int x) { return x * 2; };

// 显式指定返回类型
auto lambda4 = [](int x) -> double { return x * 1.5; };
```

5.2 捕获列表

```
cpp
```

```
int x = 10;
int y = 20;

// 按值捕获
auto lambda1 = [x](int a) { return x + a; }; // x被复制

// 按引用捕获
auto lambda2 = [&y](int a) { y += a; return y; }; // y被引用

// 捕获所有变量（按值）
auto lambda3 = [=](int a) { return x + y + a; };

// 捕获所有变量（按引用）
auto lambda4 = [&](int a) { x += a; y += a; return x + y; };

// 混合捕获
auto lambda5 = [=, &y](int a) { y += a; return x + y + a; }; // x按值, y按引用
auto lambda6 = [&, x](int a) { y += a; return x + y + a; }; // y按引用, x按值

// C++14: 初始化捕获
auto lambda7 = [z = x + y](int a) { return z + a; };
```

5.3 实际应用

STL算法中使用:

cpp

```

#include <algorithm>
#include <vector>

vector<int> vec{1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

// 查找偶数
auto it = find_if(vec.begin(), vec.end(), [](int n) { return n % 2 == 0; });

// 计算偶数个数
int count = count_if(vec.begin(), vec.end(), [](int n) { return n % 2 == 0; });

// 排序 (降序)
sort(vec.begin(), vec.end(), [](int a, int b) { return a > b; });

// 变换
vector<int> squared;
transform(vec.begin(), vec.end(), back_inserter(squared),
          [](int n) { return n * n; });

// for_each
for_each(vec.begin(), vec.end(), [](int n) { cout << n << " "; });

```

替代函数对象：

```

cpp

// 传统函数对象
class Multiplier {
private:
    int factor_;
public:
    Multiplier(int factor) : factor_(factor) {}
    int operator()(int x) const { return x * factor_; }
};

// Lambda 替代
int factor = 5;
auto multiplier = [factor](int x) { return x * factor; };

vector<int> numbers{1, 2, 3, 4, 5};
transform(numbers.begin(), numbers.end(), numbers.begin(), multiplier);

```

6. 类的新功能

6.1 委托构造函数

cpp

```
class Rectangle {
private:
    int width_, height_;

public:
    // 主构造函数
    Rectangle(int w, int h) : width_(w), height_(h) {
        cout << "Main constructor" << endl;
    }

    // 委托构造函数
    Rectangle() : Rectangle(0, 0) { // 委托给主构造函数
        cout << "Default constructor" << endl;
    }

    Rectangle(int size) : Rectangle(size, size) { // 委托给主构造函数
        cout << "Square constructor" << endl;
    }

    int area() const { return width_ * height_; }
};
```

6.2 类内成员初始化

cpp

```
class Student {
private:
    string name_ = "Unknown"; // 类内初始化
    int age_ = 0; // 类内初始化
    vector<int> grades_; // 类内初始化

public:
    Student() = default; // 使用类内初始化的默认值

    Student(const string& name) : name_(name) {
        // age_ 和 grades_ 使用类内初始化的值
    }

    Student(const string& name, int age) : name_(name), age_(age) {
        // grades_ 使用类内初始化的值
    }
};
```

6.3 default和delete

cpp

```
class MyClass {
public:
    // 显式要求编译器生成默认构造函数
    MyClass() = default;

    // 显式要求编译器生成拷贝构造函数
    MyClass(const MyClass&) = default;

    // 显式要求编译器生成移动构造函数
    MyClass(MyClass&&) = default;

    // 禁用拷贝赋值运算符
    MyClass& operator=(const MyClass&) = delete;

    // 禁用特定参数的构造函数
    MyClass(int) = delete; // 禁止从int构造

    // 析构函数
    ~MyClass() = default;
};

// 不可复制的类
class NonCopyable {
public:
    NonCopyable() = default;
    NonCopyable(const NonCopyable&) = delete;
    NonCopyable& operator=(const NonCopyable&) = delete;
    NonCopyable(NonCopyable&&) = default;
    NonCopyable& operator=(NonCopyable&&) = default;
};
```

6.4 override和final

cpp

```
class Base {
public:
    virtual void func1() {}
    virtual void func2() const {}
    virtual void func3() final {} // 不能被重写
};

class Derived : public Base {
public:
    void func1() override {} // 明确表示重写基类函数

    // void func2() override {} // 编译错误: 签名不匹配 (缺少const)
    void func2() const override {} // 正确

    // void func3() override {} // 编译错误: func3被标记为final
};

// final类不能被继承
class FinalClass final {
    // ...
};

// class CannotInherit : public FinalClass {}; // 编译错误
```

7. 模板改进

7.1 可变参数模板

```
cpp
```



```
// 递归方式处理参数包
template<typename T>
void print(T&& t) {
    cout << t << endl;
}

template<typename T, typename... Args>
void print(T&& t, Args&&... args) {
    cout << t << " ";
    print(forward<Args>(args)...);
}

// 使用
print(1, 2.5, "hello", 'c'); // 输出: 1 2.5 hello c

// C++17折叠表达式简化
template<typename... Args>
void print_cpp17(Args&&... args) {
    ((cout << args << " ", ...);
    cout << endl;
}
```

计算参数包大小:

```
cpp

template<typename... Args>
void func(Args... args) {
    cout << "参数个数: " << sizeof...(args) << endl;
    cout << "参数类型个数: " << sizeof...(Args) << endl;
}

func(1, 2.5, "hello"); // 输出: 参数个数: 3, 参数类型个数: 3
```

7.2 模板别名

```
cpp
```

```

// 传统typedef的限制
typedef vector<int> IntVec;
// typedef vector<T> GenericVec; // 错误, T未定义

// C++11模板别名
template<typename T>
using Vec = vector<T>;

template<typename T>
using Ptr = unique_ptr<T>;

// 使用
Vec<int> nums{1, 2, 3};
Ptr<int> p = make_unique<int>(42);

// 复杂别名
template<typename Key, typename Value>
using Map = unordered_map<Key, Value>;

Map<string, int> wordCount;

```

7.3 返回类型后置

```

cpp

// 传统方式的问题
template<typename T, typename U>
??? add(T t, U u) { // 返回类型难以确定
    return t + u;
}

// C++11解决方案
template<typename T, typename U>
auto add(T t, U u) -> decltype(t + u) {
    return t + u;
}

// C++14进一步简化
template<typename T, typename U>
auto add_cpp14(T t, U u) {
    return t + u; // 自动推导
}

```

8. STL容器和算法增强

8.1 新容器

array - 固定大小数组：

```
cpp

#include <array>

array<int, 5> arr{1, 2, 3, 4, 5};

// 获取大小
cout << arr.size() << endl; // 5

// 访问元素
arr[0] = 10;
arr.at(1) = 20;

// 迭代
for (const auto& elem : arr) {
    cout << elem << " ";
}

// 与STL算法配合
sort(arr.begin(), arr.end());
```

unordered_map - 哈希表：

```
cpp
```

```
#include <unordered_map>

unordered_map<string, int> wordCount;

// 插入
wordCount["hello"] = 1;
wordCount.insert({"world", 2});
wordCount.emplace("cpp", 3);

// 查找
auto it = wordCount.find("hello");
if (it != wordCount.end()) {
    cout << it->first << ": " << it->second << endl;
}

// 遍历
for (const auto& pair : wordCount) {
    cout << pair.first << ": " << pair.second << endl;
}
```

8.2 基于范围的for循环

cpp

```

vector<int> vec{1, 2, 3, 4, 5};

// 只读遍历
for (const auto& elem : vec) {
    cout << elem << " ";
}

// 修改元素
for (auto& elem : vec) {
    elem *= 2;
}

// 拷贝遍历 (通常不推荐)
for (auto elem : vec) {
    elem *= 2; // 只修改拷贝, 不影响原容器
}

// 遍历map
map<string, int> m{{"a", 1}, {"b", 2}};
for (const auto& [key, value] : m) { // C++17结构化绑定
    cout << key << ": " << value << endl;
}

```

8.3 新算法

```

cpp

#include <algorithm>

vector<int> vec{1, 2, 3, 4, 5};

// all_of, any_of, none_of
bool all_positive = all_of(vec.begin(), vec.end(), [](int x) { return x > 0; });
bool any_even = any_of(vec.begin(), vec.end(), [](int x) { return x % 2 == 0; });
bool none_negative = none_of(vec.begin(), vec.end(), [](int x) { return x < 0; });

// copy_if
vector<int> evens;
copy_if(vec.begin(), vec.end(), back_inserter(evens),
        [](int x) { return x % 2 == 0; });

// iota - 填充连续值
vector<int> sequence(10);
iota(sequence.begin(), sequence.end(), 1); // 1, 2, 3, ..., 10

```

9. 并发编程基础

9.1 线程

```
cpp

#include <thread>
#include <iostream>

void hello() {
    cout << "Hello from thread!" << endl;
}

void count_to(int n) {
    for (int i = 1; i <= n; ++i) {
        cout << i << " ";
    }
    cout << endl;
}

int main() {
    // 创建线程
    thread t1(hello);
    thread t2(count_to, 5);

    // Lambda线程
    thread t3([]() {
        cout << "Lambda thread!" << endl;
    });

    // 等待线程完成
    t1.join();
    t2.join();
    t3.join();

    return 0;
}
```

9.2 互斥量

```
cpp
```

```
#include <thread>
#include <mutex>

mutex mtx;
int counter = 0;

void increment(int times) {
    for (int i = 0; i < times; ++i) {
        lock_guard<mutex> lock(mtx); // RAII 风格的锁
        ++counter;
    }
}

int main() {
    thread t1(increment, 1000);
    thread t2(increment, 1000);

    t1.join();
    t2.join();

    cout << "Counter: " << counter << endl; // 应该是2000
    return 0;
}
```

9.3 原子操作

cpp

```
#include <atomic>
#include <thread>

atomic<int> atomic_counter{0};

void atomic_increment(int times) {
    for (int i = 0; i < times; ++i) {
        atomic_counter++; // 原子操作，无需锁
    }
}

int main() {
    thread t1(atomic_increment, 1000);
    thread t2(atomic_increment, 1000);

    t1.join();
    t2.join();

    cout << "Atomic counter: " << atomic_counter.load() << endl;
    return 0;
}
```

10. 实战练习

练习1：实现一个现代化的String类

cpp


```
#include <iostream>
#include <cstring>
#include <utility>

class String {
private:
    char* data_;
    size_t size_;

public:
    // 默认构造函数
    String() : data_(nullptr), size_(0) {}

    // 从C字符串构造
    String(const char* str) {
        if (str) {
            size_ = strlen(str);
            data_ = new char[size_ + 1];
            strcpy(data_, str);
        } else {
            data_ = nullptr;
            size_ = 0;
        }
    }

    // 拷贝构造函数
    String(const String& other) {
        size_ = other.size_;
        if (other.data_) {
            data_ = new char[size_ + 1];
            strcpy(data_, other.data_);
        } else {
            data_ = nullptr;
        }
    }

    // 移动构造函数
    String(String&& other) noexcept
        : data_(other.data_), size_(other.size_) {
        other.data_ = nullptr;
        other.size_ = 0;
    }

    // 拷贝赋值运算符
    String& operator=(const String& other) {
        if (this != &other) {
```

```

        delete[] data_;
        size_ = other.size_;
        if (other.data_) {
            data_ = new char[size_ + 1];
            strcpy(data_, other.data_);
        } else {
            data_ = nullptr;
        }
    }
    return *this;
}

```

// 移动赋值运算符

```

String& operator=(String&& other) noexcept {
    if (this != &other) {
        delete[] data_;
        data_ = other.data_;
        size_ = other.size_;
        other.data_ = nullptr;
        other.size_ = 0;
    }
    return *this;
}

```

// 析构函数

```

~String() {
    delete[] data_;
}

```

// 访问函数

```

const char* c_str() const { return data_ ? data_ : ""; }
size_t size() const { return size_; }
bool empty() const { return size_ == 0; }

```

// 重载输出运算符

```

friend std::ostream& operator<<(std::ostream& os, const String& str) {
    os << str.c_str();
    return os;
}
};

```

// 测试代码

```

int main() {
    String s1("Hello");
    String s2 = s1;        // 拷贝构造
    String s3 = std::move(s1); // 移动构造
}

```

```
std::cout << "s2: " << s2 << std::endl;  
std::cout << "s3: " << s3 << std::endl;  
  
return 0;  
}
```

练习2：使用Lambda实现函数式编程风格

cpp

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <numeric>
#include <functional>

// 函数式编程工具类
template<typename Container>
class FunctionalContainer {
private:
    Container container_;

public:
    FunctionalContainer(Container c) : container_(std::move(c)) {}

    // map操作
    template<typename Func>
    auto map(Func f) -> FunctionalContainer<std::vector<decltype(f(*container_.begin()))>> {
        std::vector<decltype(f(*container_.begin()))> result;
        std::transform(container_.begin(), container_.end(),
            std::back_inserter(result), f);
        return FunctionalContainer<decltype(result)>(std::move(result));
    }

    // filter操作
    template<typename Predicate>
    FunctionalContainer filter(Predicate pred) {
        Container result;
        std::copy_if(container_.begin(), container_.end(),
            std::back_inserter(result), pred);
        return FunctionalContainer(std::move(result));
    }

    // reduce操作
    template<typename Func>
    auto reduce(Func f) -> typename Container::value_type {
        return std::accumulate(container_.begin() + 1, container_.end(),
            *container_.begin(), f);
    }

    // forEach操作
    template<typename Func>
    void forEach(Func f) {
        std::for_each(container_.begin(), container_.end(), f);
    }
}
```

```

// 获取容器
const Container& get() const { return container_; }
};

// 工厂函数
template<typename Container>
FunctionalContainer<Container> make_functional(Container c) {
    return FunctionalContainer<Container>(std::move(c));
}

// 测试代码
int main() {
    std::vector<int> numbers{1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    // 链式操作: 过滤偶数 -> 平方 -> 求和
    auto result = make_functional(numbers)
        .filter([](int x) { return x % 2 == 0; }) // 过滤偶数
        .map([](int x) { return x * x; })        // 平方
        .reduce([](int a, int b) { return a + b; }); // 求和

    std::cout << "偶数平方和: " << result << std::endl; // 2^2 + 4^2 + 6^2 + 8^2 + 10^2 = 220

    // 打印所有奇数的立方
    make_functional(numbers)
        .filter([](int x) { return x % 2 == 1; }) // 过滤奇数
        .map([](int x) { return x * x * x; })    // 立方
        .forEach([](int x) { std::cout << x << " "; }); // 打印
    std::cout << std::endl;

    return 0;
}

```

练习3：实现一个线程安全的队列

cpp

```
#include <queue>
#include <mutex>
#include <condition_variable>
#include <memory>

template<typename T>
class ThreadSafeQueue {
private:
    mutable std::mutex mtx_;
    std::queue<T> queue_;
    std::condition_variable condition_;

public:
    ThreadSafeQueue() = default;

    // 禁用拷贝构造和赋值
    ThreadSafeQueue(const ThreadSafeQueue&) = delete;
    ThreadSafeQueue& operator=(const ThreadSafeQueue&) = delete;

    // 添加元素
    void push(T item) {
        std::lock_guard<std::mutex> lock(mtx_);
        queue_.push(item);
        condition_.notify_one();
    }

    // 等待并弹出元素
    T waitAndPop() {
        std::unique_lock<std::mutex> lock(mtx_);
        while (queue_.empty()) {
            condition_.wait(lock);
        }
        T result = queue_.front();
        queue_.pop();
        return result;
    }

    // 尝试弹出元素
    bool tryPop(T& item) {
        std::lock_guard<std::mutex> lock(mtx_);
        if (queue_.empty()) {
            return false;
        }
        item = queue_.front();
        queue_.pop();
        return true;
    }
};
```

```

}

// 尝试弹出元素 (返回智能指针)
std::shared_ptr<T> tryPop() {
    std::lock_guard<std::mutex> lock(mtx_);
    if (queue_.empty()) {
        return std::shared_ptr<T>();
    }
    auto result = std::make_shared<T>(queue_.front());
    queue_.pop();
    return result;
}

// 检查是否为空
bool empty() const {
    std::lock_guard<std::mutex> lock(mtx_);
    return queue_.empty();
}

// 获取大小
size_t size() const {
    std::lock_guard<std::mutex> lock(mtx_);
    return queue_.size();
}
};

// 测试代码
#include <thread>
#include <iostream>
#include <chrono>

int main() {
    ThreadSafeQueue<int> queue;

    // 生产者线程
    std::thread producer([&queue]() {
        for (int i = 0; i < 10; ++i) {
            queue.push(i);
            std::cout << "生产: " << i << std::endl;
            std::this_thread::sleep_for(std::chrono::milliseconds(100));
        }
    });

    // 消费者线程
    std::thread consumer([&queue]() {
        for (int i = 0; i < 10; ++i) {
            int value = queue.waitAndPop();

```

```
        std::cout << "消费: " << value << std::endl;
    }
});

producer.join();
consumer.join();

return 0;
}
```

练习4：实现一个支持链式调用的计算器

cpp


```
#include <iostream>
#include <functional>
#include <vector>
#include <sstream>

class Calculator {
private:
    double value_;
    std::vector<std::string> operations_;

public:
    Calculator(double initial = 0) : value_(initial) {
        operations_.push_back("初始值: " + std::to_string(initial));
    }

    // 加法
    Calculator& add(double x) {
        value_ += x;
        operations_.push_back("+ " + std::to_string(x));
        return *this;
    }

    // 减法
    Calculator& subtract(double x) {
        value_ -= x;
        operations_.push_back("- " + std::to_string(x));
        return *this;
    }

    // 乘法
    Calculator& multiply(double x) {
        value_ *= x;
        operations_.push_back("* " + std::to_string(x));
        return *this;
    }

    // 除法
    Calculator& divide(double x) {
        if (x != 0) {
            value_ /= x;
            operations_.push_back("/ " + std::to_string(x));
        } else {
            operations_.push_back("除零错误!");
        }
        return *this;
    }
}
```

// 自定义操作

```
Calculator& apply(std::function<double(double)> func, const std::string& desc = "自定义操作") {  
    value_ = func(value_);  
    operations_.push_back(desc);  
    return *this;  
}
```

// 条件操作

```
Calculator& if_condition(bool condition, std::function<Calculator&(Calculator&)> operation) {  
    if (condition) {  
        return operation(*this);  
    }  
    return *this;  
}
```

// 获取结果

```
double result() const {  
    return value_;  
}
```

// 重置

```
Calculator& reset(double initial = 0) {  
    value_ = initial;  
    operations_.clear();  
    operations_.push_back("重置为: " + std::to_string(initial));  
    return *this;  
}
```

// 显示计算过程

```
void showSteps() const {  
    std::cout << "计算过程:" << std::endl;  
    for (const auto& op : operations_) {  
        std::cout << " " << op << std::endl;  
    }  
    std::cout << "最终结果: " << value_ << std::endl;  
}
```

// 隐式转换为double

```
operator double() const {  
    return value_;  
}  
};
```

// 测试代码

```
int main() {  
    Calculator calc(10);
```

// 链式调用

```
double result = calc
    .add(5)           // 10 + 5 = 15
    .multiply(2)      // 15 * 2 = 30
    .subtract(10)     // 30 - 10 = 20
    .divide(4)        // 20 / 4 = 5
    .apply([](double x) { return x * x; }, "平方") // 5^2 = 25
    .if_condition(true, [](Calculator& c) -> Calculator& {
        return c.add(75); // 25 + 75 = 100 (条件为真时执行)
    })
    .result();
```

calc.showSteps();

std::cout << "最终结果: " << result << std::endl;

// 重置并进行新计算

```
calc.reset(1)
    .add(2)
    .multiply(3)
    .showSteps();
```

return 0;

}

练习5：实现一个现代化的观察者模式

cpp

```
#include <iostream>
#include <vector>
#include <functional>
#include <memory>
#include <algorithm>
#include <string>

// 事件数据基类
class EventData {
public:
    virtual ~EventData() = default;
};

// 具体事件数据
class StringEventData : public EventData {
private:
    std::string data_;
public:
    StringEventData(const std::string& data) : data_(data) {}
    const std::string& getData() const { return data_; }
};

class IntEventData : public EventData {
private:
    int data_;
public:
    IntEventData(int data) : data_(data) {}
    int getData() const { return data_; }
};

// 观察者接口
template<typename EventType>
class Observer {
public:
    virtual ~Observer() = default;
    virtual void onEvent(const EventType& event) = 0;
};

// 可观察对象
template<typename EventType>
class Observable {
private:
    std::vector<std::weak_ptr<Observer<EventType>>> observers_;
    std::vector<std::function<void(const EventType&)>> lambdaObservers_;

public:
```

```
// 添加观察者
```

```
void addObserver(std::shared_ptr<Observer<EventType>> observer) {  
    observers_.push_back(observer);  
}
```

```
// 添加Lambda观察者
```

```
void addObserver(std::function<void(const EventType&> observer) {  
    lambdaObservers_.push_back(observer);  
}
```

```
// 移除过期的观察者
```

```
void cleanupObservers() {  
    observers_.erase(  
        std::remove_if(observers_.begin(), observers_.end(),  
            [](const std::weak_ptr<Observer<EventType>>& wp) {  
                return wp.expired();  
            }  
        ),  
        observers_.end()  
    );  
}
```

```
// 通知所有观察者
```

```
void notifyObservers(const EventType& event) {  
    // 通知对象观察者  
    for (auto& weakObs : observers_) {  
        if (auto obs = weakObs.lock()) {  
            obs->onEvent(event);  
        }  
    }  
}
```

```
// 通知Lambda观察者
```

```
for (auto& obs : lambdaObservers_) {  
    obs(event);  
}
```

```
// 清理过期观察者
```

```
cleanupObservers();  
}
```

```
};
```

```
// 具体的可观察对象
```

```
class NewsPublisher : public Observable<StringEventData> {  
private:  
    std::string name_;  
  
public:  
    NewsPublisher(const std::string& name) : name_(name) {}  
};
```

```

void publishNews(const std::string& news) {
    std::cout << "[" << name_ << "]" 发布新闻: " << news << std::endl;
    notifyObservers(StringEventData(news));
}

const std::string& getName() const { return name_; }
};

// 具体观察者
class NewsSubscriber : public Observer<StringEventData> {
private:
    std::string name_;

public:
    NewsSubscriber(const std::string& name) : name_(name) {}

    void onEvent(const StringEventData& event) override {
        std::cout << "[订阅者 " << name_ << "]" 收到新闻: " << event.getData() << std::endl;
    }

    const std::string& getName() const { return name_; }
};

// 测试代码
int main() {
    // 创建发布者
    NewsPublisher publisher("科技日报");

    // 创建观察者
    auto subscriber1 = std::make_shared<NewsSubscriber>("张三");
    auto subscriber2 = std::make_shared<NewsSubscriber>("李四");

    // 添加观察者
    publisher.addObserver(subscriber1);
    publisher.addObserver(subscriber2);

    // 添加Lambda观察者
    publisher.addObserver([](const StringEventData& event) {
        std::cout << "[Lambda观察者] 自动存档: " << event.getData() << std::endl;
    });

    // 发布新闻
    publisher.publishNews("C++11标准正式发布!");
    std::cout << "---" << std::endl;

    // 移除一个观察者

```

```
subscriber1.reset();

publisher.publishNews("Lambda表达式成为热门特性!");
std::cout << "---" << std::endl;

return 0;
}
```

学习路径建议

初学者路径（第1-2周）

1. 基础语法增强

- 统一初始化：每天练习用{}初始化不同类型
- nullptr：将现有代码中的NULL改为nullptr
- 强类型枚举：重写现有的enum

2. 自动类型推导

- auto：从简单的类型开始，逐步到复杂类型
- decltype：理解其规则，特别注意括号的影响

进阶路径（第3-4周）

3. 智能指针

- 先掌握unique_ptr，理解RAII
- 再学习shared_ptr，注意循环引用问题
- 实践：将手动new/delete的代码改为智能指针

4. Lambda表达式

- 从简单的无参数Lambda开始
- 逐步学习捕获列表的各种用法
- 结合STL算法使用

高级路径（第5-6周）

5. 移动语义

- 理解左值右值的概念
- 实现移动构造函数和移动赋值
- 学会使用std::move

6. 模板和并发

- 可变参数模板的递归实现
- 基础的线程和同步原语

实战建议

1. **每日练习**：选择一个小项目，每天用一个新特性改进它
2. **代码审查**：将老代码用C++11特性重写
3. **性能测试**：比较移动语义和拷贝语义的性能差异
4. **多线程项目**：实现一个简单的生产者-消费者模型

常见陷阱

1. **auto的过度使用**：在类型不明确时要谨慎使用
2. **Lambda捕获**：注意按值和按引用捕获的生命周期问题
3. **移动语义**：move后的对象不要再使用
4. **线程安全**：shared_ptr的引用计数是线程安全的，但对象本身不是

延伸阅读

- 《有效的现代 C++》 - Scott Meyers
- 《C++ 并发在行动》 - Anthony Williams
- 《现代C++设计》 - Andrei Alexandrescu

通过系统地学习这些特性并完成实战练习，您将能够熟练使用C++11的新特性，编写更现代、更高效的C++代码。